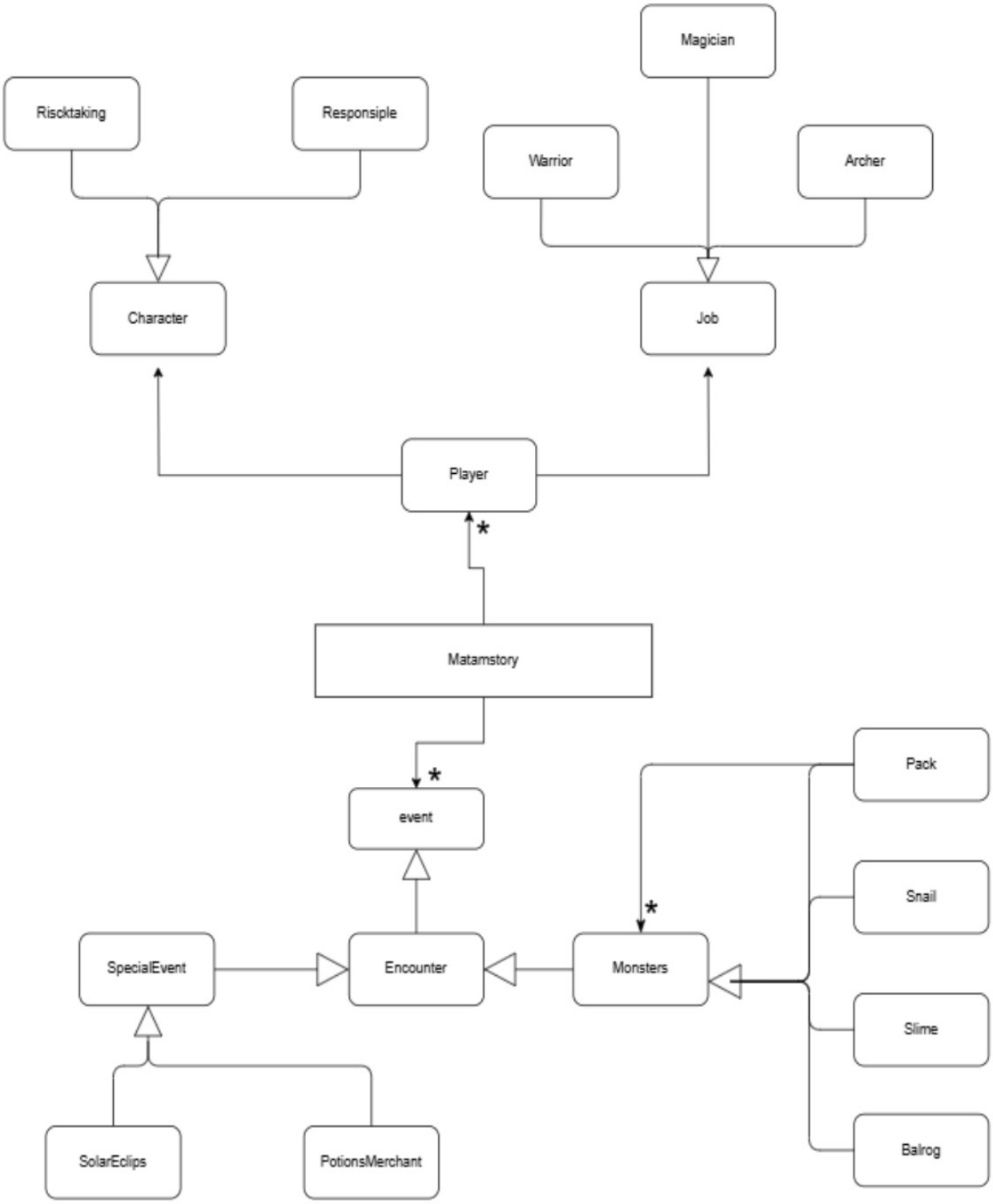
Dry section:

1. UML:



2. Design Patterns we used in our code:

1.Factory Pattern:

In our code we used factory pattern in the **MatamStory** constructor, where it creates different types of Events (Monsters) based on its type that read from the file of events (**Snail**, **Slime**, **Balrog**, **Pack**…) Each card type is created using a factory method, which creates specific objects based on the card type.

2. Composite Pattern:

We used this pattern in the **Pack** class, which contains a collection of objects from the **Monsters** class. The ability to recursively combine multiple monsters and treat them as a single large object (the pack) fits the Composite Pattern, where individual objects and groups of objects are treated in the same way.

3. Strategy Pattern:

It's shown in **Jop** class and its derived classes (**Warrior, Archer, Magician**) represent different strategies for a player's job. We used "The getstrategy()" method defines the strategy used by the player for combat. This pattern allows the Player class to use different strategies without modifying its own code.

3. actually, this is how we can implement the new " **Rogue**" job in your game, including the ability for a **Rogue** character to avoid battles with monsters whose combat power is twice or more than the **Rogue**:

1. Add the "Rogue" Class:

First, we have to create a new class **Rogue** that inherits from **Job** (because its kind of job). This class should implement the specific behavior for **Rogue** character, like avoiding battles with monsters whose combat power is twice or more than the rogue. For example:

```
Class Rogue : public Job{
///the rest of the code
};
```

2. Update game initialization:

Because when we added additional job and the player can pick it as his job, we have to modify the game's initialization process to handle the Rogue job when reading player data. All this process happens in "MtmStory file" that reads files and handle the content of the game.

3. Adjust player data file and Modify the Player Class:

We have to ensure that the player data file includes details for the **Rogue** job. We have to update the file format and content to incorporate the new job type and its attributes. And
Update the **Player class** to handle the **Rogue** job. Implement a method to check if the **Rogue** should avoid a battle based on the combat power of the monster and the player, Also in the checking input process we have to handle it in case we have Rogue job for player.

4. Modify the function that manages the game and the game Loop:

In our code we have "**playTurn()"** function that manages the rounds and the game between the players each one for his turn ,we have to modify this function for handling method,by determine if a Rogue character should avoid the battle or not, by checking the compatpower and manage the game according to this consequence.
And we have to verify that the battle avoidance (if its happen or not) is included in the **game loop** so that it is applied during each turn of the game,this is by modifying the game loop.

4. Yes, it is possible to implement the new event type "DivineInspiration" in our system. But we have to do few steps to make this happen like:

1.Create the DivineInspiration event class:

We have to create a new class for the DivineInspiration event, inheriting from the base “SpecialEvent” class that in our code. This class will handle the logic for replacing the current job with a new random job.

2.Update Player initialization

We should modify the player initialization to ensure that players are initialized with valid jobs. Also we have to check if the job is valid by checking input functions.

3. modify the function that checks the events and manage the game:

In our code the function “playEncounter” (that in player.h file), manages the game according to the type of the event, so that each type has its own attribute, so we have to **modify** this function top handle the "DivineInspiration" special event card that when this card is played, replace the player's current job with a new random job.

4. Event handling:

Within the MtmStory class, when player(character) experience the next event on the list, we have to verify if it is a "DivineInspiration"event. If it is, apply its functionality to replace the player’s existing job with a random job.

5. Handle Randomization:

if the player faced a DivineInspiration event, he has to change his job with another random job, so we have to create a method to generate a random job and return it to the player.

6. checking process:

When we add new feature in our code like the DivineInspiration event, that changes a player's job randomly ,we have to ensure that this thing will not affects the game and its rules that’s why we must create validation functions to check that the new job assignment from the DivineInspiration event does not break any rules.